

Documentation

de

l'application

Stubborn

1. Présentation

Stubborn est une application e-commerce développée avec Symfony.

Fonctionnalités principales :

- authentification (inscription, connexion, déconnexion),
- rôles utilisateur (`ROLE\_CLIENT` , `ROLE\_ADMIN`),
- affichage catalogue + fiche produit,
- ajout au panier avec taille,
- création de commande,
- paiement Stripe (mode simulation ou mode test réel),
- back-office admin (CRUD produits),
- upload d'images produits.

2. Stack technique

- **Backend** : Symfony 7
- **Langage** : PHP 8.2
- **Base de données** : MySQL/MariaDB (XAMPP)
- **ORM** : Doctrine
- **Templating** : Twig
- **Styles** : CSS natif (`public/styles/app.css`)
- **Paiement** : Stripe PHP SDK
- **Tests** : PHPUnit

3. Structure du projet

Dossiers importants :

- `src/Controller`` : contrôleurs web
- `src/Entity`` : modèles Doctrine
- `src/Form`` : formulaires Symfony
- `src/Service`` : logique métier (panier, Stripe)
- `src/Repository`` : requêtes base de données
- `src/DataFixtures`` : données de démo
- `templates`` : vues Twig
- `public/images/products`` : images des produits
- `config/packages/security.yaml`` : règles de sécurité

4. Modèle de données

Entités principales :

- `User`
 - infos utilisateur (email, mot de passe hashé, nom, adresse)
 - rôles (`ROLE\_CLIENT`, `ROLE\_ADMIN`)
 - statut de vérification email (`isVerified`)
- `Product`
 - nom, prix, image, flag `featured`
 - stocks par taille (`XS`, `S`, `M`, `L`, `XL`)
- `CustomerOrder`
 - utilisateur propriétaire
 - total, statut (`pending` / `paid`), identifiant de session Stripe
 - relation vers lignes de commande
- `OrderItem`
 - produit lié
 - snapshot métier : nom produit, prix unitaire, taille, quantité

5. Règles de sécurité et accès

Configuration : ``config/packages/security.yaml``.

Accès contrôlé par rôle :

- ``/admin`` → ``ROLE_ADMIN``
- ``/products``, ``/product/*``, ``/cart/*`` → ``ROLE_CLIENT``

Conséquences :

- un admin peut aussi agir comme client si son rôle contient ``ROLE_CLIENT``.
- un utilisateur non connecté ne peut pas commander.

6. Parcours fonctionnels

6.1 Parcours client :

1. Le client se connecte (ou s'inscrit).
2. Il ouvre le catalogue (`/products`).
3. Il consulte une fiche produit (`/product/{id}`).
4. Il choisit une taille et ajoute au panier.
5. Il ouvre le panier (`/cart`).
6. Il lance le checkout (`/cart/checkout`).
7. Au succès (`/cart/success/{orderId}`), la commande passe `paid` et le panier est vidé.

6.2 Parcours admin :

1. L'admin se connecte.
2. Il ouvre le back-office (`/admin`).
3. Il peut :
 - créer un produit,
 - modifier un produit,
 - supprimer un produit.

6.3 Inscription et vérification email :

1. Un utilisateur s'inscrit via `/register` .
2. Le mot de passe est hashé.
3. Le rôle `ROLE\_CLIENT` est attribué.
4. Un email de confirmation est généré avec lien signé.
5. Le lien `/verify/email` active le compte (`isVerified = true`).

7. Gestion du panier et des commandes

Panier :

La logique panier est portée par `CartService`.

Responsabilités :

- ajouter une ligne (produit + taille),
- supprimer une ligne,
- calculer le total,
- retourner les lignes détaillées,
- vider le panier.

Commande :

Dans `CartController::checkout` :

- le panier est validé (non vide),
- une `CustomerOrder` est créée,
- les `OrderItem` sont créés en snapshot,
- la session Stripe est créée,
- l'ID Stripe est sauvegardé en base.

8. Intégration Stripe

Service : `StripeService`.

Deux modes :

- ****Simulation locale**** si `STRIPE\_SECRET\_KEY=sk\_test\_\*\*\*`
- ****Stripe test réel**** si clé `sk\_test\_...` valide

Flux :

1. génération URL de succès/annulation,
2. création session checkout,
3. redirection vers Stripe,
4. retour succès vers `/cart/success/{orderId}`.

Carte de test Stripe utilisée en démo :

« 4242 4242 4242 4242 »

9. Upload d'images produits

Implémentation dans ``AdminController``.

Comportement :

- le formulaire admin accepte un fichier image,
- le nom de fichier est nettoyé + rendu unique,
- l'image est déplacée dans ``public/images/products/``,
- le chemin web est sauvegardé dans ``Product::imagePath``.

Contraintes usuelles du formulaire :

- formats : ``jpg``, ``jpeg``, ``png``, ``webp``,
- taille max : 2 Mo.

10. Fixtures (données de démonstration)

Fichier : `src/DataFixtures/AppFixtures.php` .

Contenu :

- 1 compte admin
- 1 compte client
- 10 produits de démonstration

Comptes fournis :

- admin : `admin@stubborn.local` / `admin123`
- client : `client@stubborn.local` / `client123`

11. Installation et lancement (résumé technique)

Depuis la racine du projet :

Powershell :

```
$php='C:/Users/darkl/Downloads/xampp-windows-x64-8.2.12-0-VS16/xampp/php/php.exe'
```

```
& $php composer.phar install
```

```
& $php bin/console doctrine:database:create --if-not-exists
```

```
& $php bin/console doctrine:schema:update --force
```

```
& $php bin/console doctrine:fixtures:load --no-interaction
```

```
& $php -S 127.0.0.1:8000 -t public
```

URL locale : `http://127.0.0.1:8000`

12. Tests

Commande :

Powershell :

```
$php='C:/Users/darkl/Downloads/xampp-windows-x64-8.2.12-0-  
VS16/xampp/php/php.exe'  
  
& $php bin/phpunit
```

Objectif :

Vérifier les comportements essentiels (panier / Stripe simulation).

13. Points d'amélioration possibles

- gestion avancée du stock (décrément au paiement),
- historique de commandes côté client,
- webhooks Stripe pour fiabiliser le statut payé,
- pagination/recherche catalogue,
- monitoring/logs dédiés production,
- pipeline CI/CD.

14. Références de fichiers utiles

- Contrôleurs : `src/Controller`
- Entités : `src/Entity`
- Sécurité : `config/packages/security.yaml`
- Services : `src/Service/CartService.php`, `src/Service/StripeService.php`
- Fixtures : `src/DataFixtures/AppFixtures.php`
- Vues : `templates/`
- Styles : `public/styles/app.css`